

AQuery and KDB-X with Docker

The course AQuery workflow works with Mac M-series Docker and x86 Linux. See [platform support](#) for the tested hosts.

What the tools do

AQuery is a compiler. It reads an AQuery source file with the `.a` extension and generates a q program with the `.q` extension. KDB-X provides the q runtime that executes the generated program.

```
AQuery source      AQuery compiler      q program      KDB-X
program.a          a2q                  program.q      q program.q
->                ->                ->
```

These notes use the AQuery compiler from <https://github.com/josepablocam/aquery>. Do not use the unrelated AQuery2 prompt .py project described in older course slides.

Every command in these notes runs AQuery and KDB-X inside the course Docker image. Students do not install Java, Scala, AQuery, PyKX, or q directly on the host.

The course image contains x86 Linux binaries for AMD64.

The Mac M-series Docker path passed on an M1 Mac with Docker Desktop 4.87. Native KDB-X and AQuery also worked on Mac M-series, but the course uses Docker so students follow the same commands.

Requirements

- Docker Engine on x86 Linux, or Mac M-series Docker
- Git
- A KX Developer account and Community license

Check Docker before continuing:

```
docker version
docker info
```

Both commands must report a running Docker server. With Mac M-series Docker, the server architecture may be ARM64. The course commands explicitly request the `linux/amd64` image.

Get the course repository

```
git clone https://github.com/gpu004/advance-database.git
cd advance-database
```

Run the remaining commands from the repository root.

Get a KDB-X Community license

- 1 Open the official KDB-X installation page: https://code.kx.com/kdb-x/get_started/kdb-x-install.html
- 2 Sign in or create a KX Developer account.
- 3 Obtain the complete base64-encoded Community license value.
- 4 Keep the license private. Do not place it in Git, screenshots, lecture notes, or shared chat messages.

Create the local environment file:

```
cp .env.example .env
chmod 600 .env
```

Edit `.env` and replace the example value:

```
KDB_LICENSE_B64=paste_the_complete_base64_license_value_here
```

Confirm that Git does not include the file:

```
git status --short
```

Do not print the license while asking for help.

Build the course image

```
docker build --platform=linux/amd64 -t aquery:linux-x86 \
  -f aquery/docker-x86-linux/Dockerfile aquery
```

The first build downloads the pinned base image, KDB-X, AQuery source, Scala dependencies, and PyKX. It is slower with Mac M-series Docker because Docker emulates an AMD64 processor.

Confirm the image architecture:

```
docker image inspect aquery:linux-x86 \
  --format 'os={{.Os}} architecture={{.Architecture}}'
```

Expected result:

```
os=linux architecture=amd64
```

Start q

```
docker run --rm -it --platform=linux/amd64 \
  --env-file .env aquery:linux-x86
```

At the q prompt, run a smoke test:

```
q)1+1
2
```

Enter a single backslash to exit q:

```
q)\
```

Run a few q queries

Start q again, then create an in-memory table:

```
q)sales:([ item:`apple`banana`coffee; amount:3 7 12])
```

Display the table:

```
q)select from sales
```

Count the rows:

```
q)count sales
3
```

Show the items whose amount is greater than 5:

```
q)select from sales where amount>5
```

The last query returns banana with amount 7 and coffee with amount 12.

Compile and run the included AQuery example

The example directory contains `sales.csv` and `simple_sales.a`. The query loads three rows and selects amounts greater than 5.

Remove an older generated file, if present:

```
rm -f example/simple_sales/simple_sales.generated.q
```

Compile the AQuery file into q:

```
docker run --rm --platform=linux/amd64 --env-file .env \
-v "$PWD/example/simple_sales:/work" aquery:linux-x86 \
a2q -c -a 1 -o simple_sales.generated.q simple_sales.a
```

Confirm that the compiler created the file:

```
ls -lh example/simple_sales/simple_sales.generated.q
```

Run the generated program with KDB-X:

```
docker run --rm --platform=linux/amd64 --env-file .env \
-v "$PWD/example/simple_sales:/work" aquery:linux-x86 \
q simple_sales.generated.q
```

The result contains:

```
item    amount
banana  7
coffee 12
```

The Docker bind mount maps the host example directory to /work in the container. The generated q file remains on the host after the container exits.

Run your own AQuery file

Place the .a file and its input data in one directory. From that directory, run:

```
docker run --rm --platform=linux/amd64 --env-file /path/to/advance-database/.env \
-v "$PWD:/work" aquery:linux-x86 \
a2q -c -a 1 -o program.generated.q program.a
```

```
docker run --rm --platform=linux/amd64 --env-file /path/to/advance-database/.env \
-v "$PWD:/work" aquery:linux-x86 \
q program.generated.q
```

Replace /path/to/advance-database/.env with the absolute path to the private environment file.

Troubleshooting

Docker server is unavailable

If `docker version` reports that it cannot connect to the daemon, start Docker Engine or Docker Desktop and retry.

License error

Confirm that .env contains one complete base64 value and has private permissions:

```
chmod 600 .env
```

Do not paste the license into an issue or support message.

Image runs with the wrong architecture

Keep `--platform=linux/amd64` on both `docker build` and `docker run`. The image contains AMD64 binaries.

Bind mount is empty with Mac M-series Docker

Open Docker Desktop's file-sharing settings and confirm that the repository directory is shared. Then rerun the command from the repository root.

AQuery cannot find a CSV file

Mount the directory containing both the `.a` file and its data to `/work`. Refer to the data with a path relative to that directory.

Build fails while downloading dependencies

Record the complete `docker build` output. Do not replace the pinned image digests or AQuery revision with unverified versions.

Last verified August 30, 2026. See the [validation record](#).

References

- KDB-X installation and licensing: https://code.kx.com/kdb-x/get_started/kdb-x-install.html
- AQuery source: <https://github.com/josepablocam/aquery>
- Docker Desktop: <https://docs.docker.com/desktop/>
- Course repository: <https://github.com/gpu004/advance-database>